Porting a Precise Point Positioning Method to RTKLIB

Technical synthesis of the methodological transfer and the RTKLIB code evolutions

Generated with ChatGPT 5.4

March 27, 2026

Abstract

This report summarizes the port of a Precise Point Positioning (PPP) method into a dedicated RTKLIB branch. The objective of the port is not to replace RTKLIB's estimator architecture, but to extend it so that the method described in the companion offline and real-time PPP papers can be reproduced within an open and reusable software framework. The resulting branch keeps RTKLIB's overall workflow, input/output philosophy, and command-line processing model, while introducing substantial changes in the PPP engine itself: true multi-frequency handling, code-to-frequency consistency across measurement modeling and ANTEX corrections, explicit support for SINEX observable-specific biases, Doppler-aided dynamics with receiver clock-drift states, generalized cycle-slip monitoring, and a new partial ambiguity-resolution module based on single-difference wide-lane fixing. The attached Septentrio post-processed runs indicate horizontal RMS values in the 5–8 cm range for East and North components, which is consistent with the intended centimeter-class trajectory reconstruction objective in constrained conditions.

1 Context and objective

The two companion papers frame the target application: centimeter-class PPP trajectory reconstruction, first in post-processing and then in a real-time perspective [1, 2]. In both cases, the core challenge is the same: to preserve PPP's infrastructure-light nature while making it robust enough for constrained environments where signal masking, multipath, intermittent slips, constellation asymmetries, and observable-specific biases strongly affect performance.

The RTKLIB branch used for the port is a public fork of RTKLIB, on branch `rtklib_2.4.3_b34_ppp` [3]. GitHub identifies it as a fork of the official `tomojitakasu/RTKLIB` repository, and the branch page shows 133 commits at the time of inspection [3]. RTKLIB itself is an open-source GNSS positioning package supporting both real-time and post-processing workflows [4, 5], and the `rnx2rtkp` application is the standard command-line entry point for computing solutions from RINEX, SP3, CLK, and related products [6].

The practical goal of the port can therefore be stated as follows:

- preserve RTKLIB's reusable solver infrastructure and user workflow;
- transplant the methodological ingredients required by the PPP approach into RTKLIB's PPP module;
- keep the implementation compatible with both post-processed and real-time use cases;
- expose the new capabilities through explicit source-level and option-level changes instead of ad hoc external scripts.

2 Methodological continuity with the reference papers

Even without rewriting the original papers, their common technical thread is clear from their scope and from the resulting implementation:

- a single-receiver PPP architecture aimed at centimeter-level kinematic trajectory reconstruction;
- operation in constrained conditions, where the error budget is dominated less by pure open-sky convergence than by robustness to stressed observations;
- strong reliance on carrier phase, but with explicit support functions for bias handling, slip control, and model consistency;
- a progression from offline reconstruction toward real-time-capable operation;
- a preference for a modular GNSS estimator rather than a narrowly tailored experimental script.

This leads naturally to a design philosophy in the RTKLIB port: the port must not merely "enable PPP" in stock RTKLIB, but must reshape the estimator so that multi-frequency, bias-aware, high-rate, and ambiguity-capable processing becomes native in the PPP engine.

3 Baseline RTKLIB architecture and porting strategy

Stock RTKLIB already provides the essential scaffolding: GNSS observation parsing, precise products ingestion, PPP processing modes, command-line execution through `rnx2rtkp`, and the general state-space Kalman filtering structure [5, 6]. However, the custom branch shows that the method could not be ported by parameter tuning alone. The source history embedded in the code makes this explicit.

In `src/ppp.c`, the custom history adds two key milestones after the upstream 2020 entry:

- `2026/03/16`: "full multi-frequency support (receiver DCBs in state vector), correct bugs in PCOs and PCVs";
- `2026/03/18`: "add Doppler observations in PPP with receiver clock-drift states".

In `src/rtkcmn.c`, another custom history entry is added:

- `2026/03/20`: "frequency plan and added consistency with ANTEX PCO/PCV".

In `src/options.c`, the custom history adds:

- `2026/03/20`: "add `file-snxbiasfile`".

These code-level milestones already summarize the porting strategy: preserve RTKLIB's global architecture, but change the PPP state definition, the observable modeling, the antenna/frequency mapping, the ambiguity engine, and the external product interface.

4 Main RTKLIB evolutions introduced by the port

4.1 From stock PPP to true multi-frequency PPP

A first major difference appears in the state vector. In upstream RTKLIB, the PPP state is essentially limited to receiver kinematics, clock states, troposphere, and one ambiguity state per satellite. In the custom branch, the macro block in `src/ppp.c` is expanded to

$$x = \begin{bmatrix} \text{position/velocity/acceleration} \\ \text{receiver clocks} \\ \text{receiver clock drift} \\ \text{troposphere} \\ \text{ionosphere states} \\ \text{receiver DCB states} \\ \text{ambiguities for each satellite and frequency} \end{bmatrix}.$$

Concretely, the branch introduces

$$\texttt{NCD(opt)} \quad \text{for clock-drift states,}$$

$$\texttt{NI(opt)} \quad \text{for estimated ionosphere states,}$$

$$\texttt{ND(opt)} \quad \text{for receiver differential code-bias states,}$$

and

$$\texttt{NB(opt)} = \texttt{NF(opt)} \times \texttt{MAXSAT}$$

for frequency-dependent ambiguity states.

This is a structural change, not a cosmetic one. It means the implementation moves away from the narrow interpretation of PPP as an ionosphere-free dual-frequency chain and toward a genuine multi-frequency, uncombined-capable architecture. The code still preserves the ionosphere-free mode as a compatibility and fallback path, but the estimator is no longer designed around that single mode.

4.2 Consistent frequency plan and ANTEX compatibility

A second major change concerns observation-to-frequency consistency. In the custom `src/rtkcmn.c`, the branch introduces an explicit frequency-plan layer through `sigband2idx()`, `code2idx()`, `code2freq()`, and a dedicated ANTEX mapping function `antexfreq2idx()`. The history comment explicitly links this work to "frequency plan" and "consistency with ANTEX PCO/PCV".

This matters because constrained-environment PPP is extremely sensitive to subtle inconsistencies between:

- the observation code used for the carrier and code measurements;
- the internal frequency index used in the state vector;
- the antenna phase center corrections loaded from ANTEX;
- the ambiguity definition associated with each observable.

The custom branch also changes `readpcv()` so that the legacy RTKLIB fallback—copying L2 antenna corrections to higher slots when data are missing—is restricted to receiver antennas only. The source comment states that performing the same fallback for satellite antennas would break the new `code2idx()`-compatible slot semantics. This is an important design decision: the port favors semantic consistency over permissive but potentially misleading frequency fallback.

4.3 SINEX observable-specific bias support

The branch extends RTKLIB's navigation and file-option structures to support SINEX observable-specific biases (OSB). In `src/rtklib.h`:

- `filopt_t` gains a new field `snxbias`;
- `nav_t` gains a pointer `osb`;

- new APIs are declared: `readsinexbias()`, `getsinexbias()`, and `freesinexbias()`.

In `src/options.c`, the new configuration key

```
"file-snxbiasfile"
```

is added, making the bias product a first-class external file in the RTKLIB configuration model.

Inside `corr_meas()` in `src/ppp.c`, the OSB logic is not passive. The implementation applies code biases systematically when available, optionally applies phase biases depending on ambiguity-resolution mode, and uses configurable fallback masks by constellation. If a required bias is missing, the observation is skipped rather than silently used in an inconsistent way. This is precisely the kind of estimator hardening needed when the objective is reliable centimeter-class performance in stressed environments.

4.4 Doppler integration and receiver clock-drift states

The branch adds Doppler as an explicit PPP observable. This is one of the clearest departures from stock RTKLIB's PPP implementation.

The custom `src/ppp.c` introduces:

- `MEAS_DOP` as a third measurement type;
- a corrected Doppler function `corr_dop_meas()`;
- dedicated stochastic modeling through `varerr_dop()` and `varerr_dop_iflc()`;
- a receiver clock-drift state updated by `udclkd_ppp()`;
- broadcast-ephemeris satellite clock drift extraction through `satclkdrift_dop()`.

The corresponding observation model is range-rate based:

$$D_{s,f} \approx \mathbf{e}_s^T(\mathbf{v}_s - \mathbf{v}_r) + c(\dot{t}_r - \dot{t}_s) + \varepsilon_D. \tag{1}$$

This is highly relevant for the real-time and high-rate trajectory objective of the papers. Carrier phase remains the precision backbone, but Doppler strengthens short-term dynamics and improves velocity observability, especially when the environment causes rapid changes in measurement quality. The port therefore uses Doppler not as a diagnostic side channel, but as a real filter observation.

4.5 Generalized cycle-slip monitoring

The branch generalizes cycle-slip detection to the multi-frequency context. In the custom `detslp_gf()` and `detslp_mw()` functions, geometry-free (GF) and Melbourne-Wubbena (MW) combinations are evaluated between the reference frequency ($f = 0$) and all other available frequencies. This is more appropriate for a genuine multi-frequency PPP engine than a hard-coded dual-frequency assumption.

Conceptually, the logic is:

- maintain reference-frequency-centered slip observables for each satellite;
- test all available auxiliary frequencies against the same reference;
- if a slip is detected in one of these pairings, mark the corresponding satellite arc as unreliable.

In a context, this improves robustness because signal interruption is not uniform across signals and constellations. A slip strategy designed only around a fixed dual-frequency pair would leave a large part of the available redundancy unused.

4.6 New ambiguity-resolution engine

The ambiguity module is also rewritten in a methodologically meaningful way. Upstream `ppp_ar.c` is based on an older linear-combination PPP-AR formulation. The custom `ppp_ar.c` explicitly states a new two-step partial ambiguity-resolution strategy inspired by Banville et al. [7]:

1) single-difference wide-lane fixing,

2) then optional reference-frequency single-difference ambiguity fixing.

The file header highlights the main properties of the new implementation:

- explicit extraction of an active ambiguity subset (x_a, Q_a);
- `lambda()` applied on transformed ambiguity subsets;
- validation by consistency of rounded candidates;
- reinjection of accepted constraints into the global state and covariance.

This is exactly the kind of selective, partial, and robust ambiguity handling that fits kinematic PPP. Instead of forcing a fragile all-or-nothing ambiguity solution, the branch promotes ambiguity fixing to a controlled estimator constraint mechanism.

5 Observation model implemented in the branch

The resulting PPP model can be summarized as follows.

For a code observation on satellite s, frequency f:

$$P_{s,f} = \rho_s + c(dt_r - dt_s) + T_s + I_{s,f} + b^P_{r,f} + \varepsilon_P. \tag{2}$$

For a phase observation:

$$\Phi_{s,f} = \rho_s + c(dt_r - dt_s) + T_s - I_{s,f} + \lambda_f N_{s,f} + \varepsilon_\Phi. \tag{3}$$

For Doppler:

$$D_{s,f} = \dot{\rho}_s + c(\dot{t}_r - \dot{t}_s) + \varepsilon_D. \tag{4}$$

What distinguishes the port from a stock implementation is not the textbook form of these equations, but the completeness of their software realization:

- code and phase are linked to an explicit frequency plan;
- ionosphere can be either estimated explicitly or suppressed by ionosphere-free combination;
- receiver DCB states are part of the filter rather than hidden in ad hoc corrections;
- observable-specific biases are introduced from an external SINEX product;
- Doppler is integrated in the same PPP residual engine.

6 What remained unchanged

Although the changes are substantial, the port remains recognizably RTKLIB. This is important for maintainability and reproducibility.

The following elements are intentionally preserved:

- the overall Kalman-filter workflow and residual-update logic;

- the standard product ingestion model through RINEX/SP3/CLK and related files;
- the command-line processing path through `rnx2rtkp`;
- the core data structures and file-driven configuration philosophy;
- the separation between common utilities (`rtkcmn.c`), estimator logic (`ppp.c`), ambiguity logic (`ppp_ar.c`), and public declarations (`rtklib.h`).

The port should therefore be viewed as a deep estimator extension rather than a fork that abandons the original RTKLIB ecosystem.

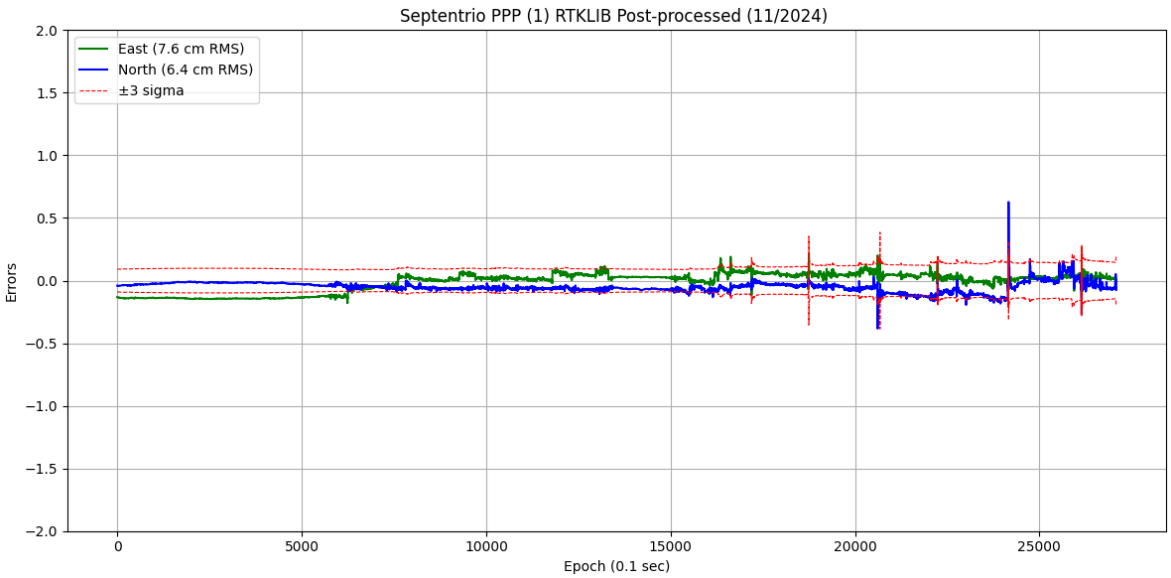
7 Experimental indication from the attached runs

Figures 1a and 1b show two attached Septentrio post-processed PPP runs, both labeled as RTKLIB post-processing in November 2024. The horizontal errors are expressed in East and North, together with a $\pm 3\sigma$ envelope.

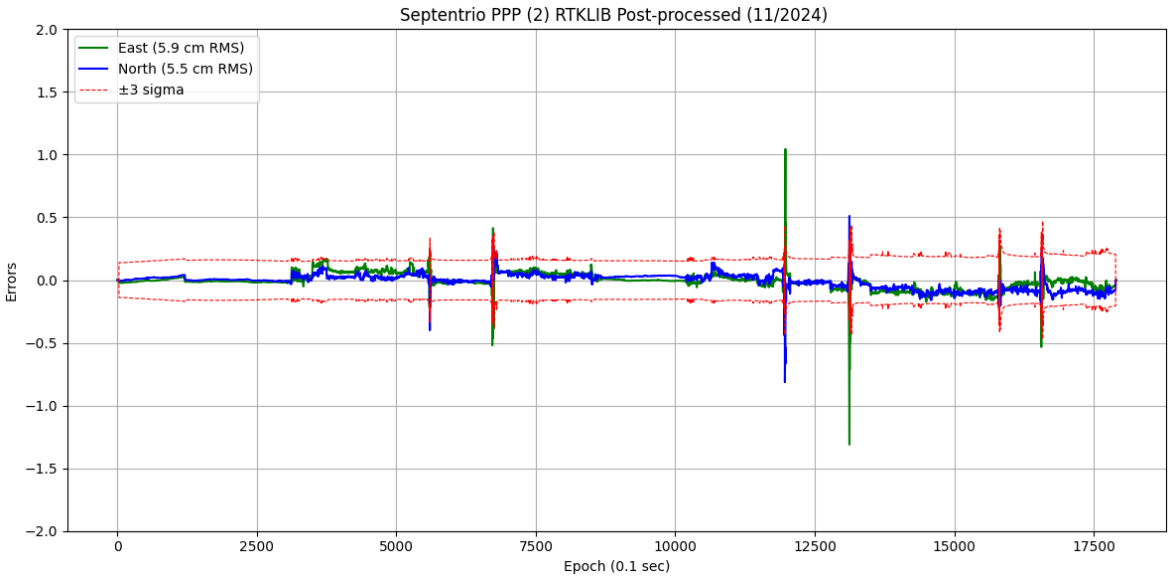
The figures support several observations:

- The branch reaches clear centimeter-class horizontal performance in both runs. The best run is close to 6 cm RMS on both East and North.
- The error sequences remain centered near zero for most of the trajectory, which suggests that the estimator is not dominated by a persistent deterministic bias.
- Most of the trajectory is contained inside the plotted $\pm 3\sigma$ bounds, which indicates that the stochastic model is broadly coherent with the observed residual behavior.
- The most difficult intervals are localized and transient. They appear as brief excursions, mainly in the later parts of the trajectories. This is the typical signature of constrained-environment segments rather than a global model failure.
- Run 2 is visibly cleaner than Run 1, both in RMS and in the relative compactness of the residual time series. This suggests that the estimator benefits strongly from observation quality and local environment, even when the processing chain is unchanged.

A useful interpretation is that the port achieves the intended balance: it does not eliminate all transient degradations, but it keeps them localized while maintaining a sub-decimeter horizontal envelope over the majority of the track.



(a) Run 1. Reported RMS: East 7.6 cm, North 6.4 cm.



(b) Run 2. Reported RMS: East 5.9 cm, North 5.5 cm.

Figure 1: Attached horizontal error plots for two Septentrio PPP runs processed with the RTKLIB branch.

8 Synthesis of the RTKLIB evolutions

Table 1 summarizes the main changes from stock RTKLIB to the custom PPP branch.

Topic	Stock RTKLIB tendency	Custom branch evolution
State vector	PPP centered on position, clocks, tropo, and one ambiguity state per satellite	Multi-frequency state structure with explicit ionosphere states, receiver DCB states, receiver clock-drift state, and ambiguities indexed by satellite and frequency
Frequency handling	Limited practical frequency semantics in PPP	Explicit frequency plan via `code2idx()`, `code2freq()`, and ANTEX-consistent slot mapping
Antenna modeling	Legacy PCV/PCO fallback behavior can blur slot semantics	Revised `readpcv()` behavior preserves receiver fallback but avoids satellite-side inconsistency with the new frequency mapping
Bias products	Standard DCB support only	Full SINEX OSB ingestion path with file option, navigation storage, lookup APIs, constellation fallback masks, and observation skipping when required biases are absent
Dynamics	Phase and code dominate PPP update	Doppler added as a native PPP observable with receiver clock-drift estimation and range-rate residuals
Cycle slips	Dual-frequency-oriented logic	GF and MW slip monitoring generalized to all available frequencies relative to the reference frequency
Ambiguity resolution	Legacy PPP-AR logic based on older linear-combination design	New partial AR engine with single-difference wide-lane fixing, transformed ambiguity subsets, and selective reinjection of validated constraints

Table 1: Main estimator-level evolutions introduced by the custom RTKLIB branch.

9 Conclusion

The dedicated RTKLIB branch can be understood as a methodical port of a centimeter-level PPP trajectory reconstruction approach into a mature open-source GNSS framework. The port keeps RTKLIB's execution model and ecosystem, but it substantially upgrades the PPP estimator itself. The essential evolutions are:

- a true multi-frequency PPP state-space formulation,
- signal- and product-consistent PCO/PCV handling,
- native support for SINEX observable-specific biases,
- Doppler-aided kinematic PPP with receiver clock-drift states,
- generalized multi-frequency cycle-slip control,
- and a new partial ambiguity-resolution engine based on single-difference wide-lane fixing.

Taken together, these changes explain why the branch is much better aligned with the requirements of high-precision trajectory reconstruction than stock RTKLIB PPP. The attached Septentrio results, with horizontal RMS values between roughly 5.5 and 7.6 cm, show that the

implementation reaches the intended centimeter-class regime while keeping the difficult events localized rather than catastrophic.

References

[1] D. Laurichesse, *Precise Point Positioning - Centimeter accuracy trajectory reconstruction in peri-urban environment*. GitHub-hosted PDF in the `navigation` repository. Available at: GitHub PDF link.

[2] D. Laurichesse, *Precise Point Positioning - Real-time centimeter accuracy trajectory in peri-urban environment*. GitHub-hosted PDF in the `navigation` repository. Available at: GitHub PDF link.

[3] D. Laurichesse, *RTKLIB*, branch `rtklib_2.4.3_b34_ppp`. GitHub repository. Available at: https://github.com/denis-laurichesse/RTKLIB/tree/rtklib_2.4.3_b34_ppp.

[4] T. Takasu, *RTKLIB: An Open Source Program Package for GNSS Positioning*. Official RTKLIB site. Available at: https://www.rtklib.com/.

[5] T. Takasu, *tomojitakasu/RTKLIB*. Official GitHub repository. Available at: https://github.com/tomojitakasu/RTKLIB.

[6] *rnx2rtkp(1) - rtklib - Debian testing manual page*. Available at: https://manpages.debian.org/testing/rtklib/rnx2rtkp.1.en.html.

[7] S. Banville et al., "Enabling ambiguity resolution in CSRS-PPP," *NAVIGATION*, vol. 68, no. 2, pp. 433–451, 2021. DOI-based landing page: https://onlinelibrary.wiley.com/doi/full/10.1002/navi.423.